



Faculty of Engineering & Technology
Electrical & Computer Engineering Department
Applied Cryptography – ENCS4320
Final Course Project – Design Report
SecureVault: A Multi-User Encrypted Document Exchange System

Prepared by:

Name: Razan Shalabi.

Number: 1230874.

PartnerOneName: Lara Daifallah.

ID_1: 1230239.

PartnerTwoName: Shatha Abualrob.

ID_2: 1231279.

Instructor: Dr. A. Shawahna.

Repository: github.com/Holmes-99/SecureVault

Date: 27/9/2026.

Abstract

SecureVault is a client–server system that lets users store private documents on a server they do not trust and share them with one another. Every document is encrypted on the owner's machine with AES-256-GCM under a fresh per-document key; that key reaches each recipient only after it has been sealed with an X25519 key agreement and signed with Ed25519, so the server stores every byte but can read none of them. Passwords are stretched with memory-hard Argon2id and never leave the client, public keys are trusted through pinning and a 20-digit safety number, and every request and document carries timestamps, nonces and version numbers that make replays detectable. All cryptographic primitives except Argon2id were implemented from scratch in Python and verified against official test vectors and a reference library. A network attacker that flips bytes, renames files, replays old versions and swaps public keys was built alongside the system; every one of its attacks is detected and rejected.

Table of Contents

Abstract	I
Table Of Figures	IV
List Of Tables	V
Introduction.....	6
Threat Model.....	6
System Design	6
• Architecture.....	6
• Key Hierarchy	7
• Byte-Level Formats	8
• Life of a Document	8
Design Decisions and Justification	9
• Protecting Passwords	9
• Protecting Document Contents	10
• Public-Key Cryptography	10
• Trusting a Public Key	11
• Signatures Versus MACs.....	11
• Freshness.....	12
• Architecture, Formats and Errors.....	12
Security Argument.....	12
• How the Mechanisms Fit Together.....	12
• Attacks Tried Against Our Own System	13
Implementation and Testing	13
Limitations	14

AI Usage Statement	15
Team Contributions	16
Conclusion	17
References.....	17

Table Of Figures

Figure 1: SecureVault architecture with the attacker proxy used in the demonstration	7
Figure 1: Key hierarchy from password to document	7
Figure 1: Producing a key grant when Layla shares a document with Omar	9
Figure 1: Verified retrieval: five ordered checks before a file is saved.....	9
Figure 1: Measured Argon2id time per hash ($t = 3$, $p = 4$)	10
Figure 1: Curve25519 versus RSA-3072 at 128-bit security (OpenSSL, median of 20 runs).....	11
Figure 1: Graphical client: safety-number check and a rejected download.....	14

List Of Tables

Table 1: Parties and their capabilities	6
Table 1: Byte-level formats of stored objects and protocol messages.....	8
Table 1: Security properties and the mechanisms that provide them	12
Table 1: Attacks run against SecureVault over real sockets.....	13
Table 1: Estimated offline attack time per stolen account.....	13

Introduction

The goal of SecureVault is to make one story true. Layla signs up and uploads thesis-draft.pdf; the server can tell its size and when it arrived, but not a word of its contents. She shares it with Omar, whose client confirms that the file is intact and that it came from Layla, and Omar can prove this to a third party. When an attacker flips a byte, renames the file, replays yesterday's upload or steals the whole database, the system either refuses the object or, in the case of the database, leaves the attacker guessing passwords for longer than anyone cares.

This report states the threat model, describes the design down to the byte level, justifies each decision against the alternatives we rejected, argues how each required security property is met, and lists what the design does not stop.

Threat Model

Table 1: Parties and their capabilities

Party	Capabilities and assumptions
Users	Run our client on a trusted machine; only they know their password.
Server	Honest-but-curious: follows the protocol and never deletes or invents data, but reads everything it stores and receives; its whole database may be stolen.
Attacker	Reads and modifies all network traffic, replays old messages, registers accounts of their own, and may obtain a full copy of the server's data.

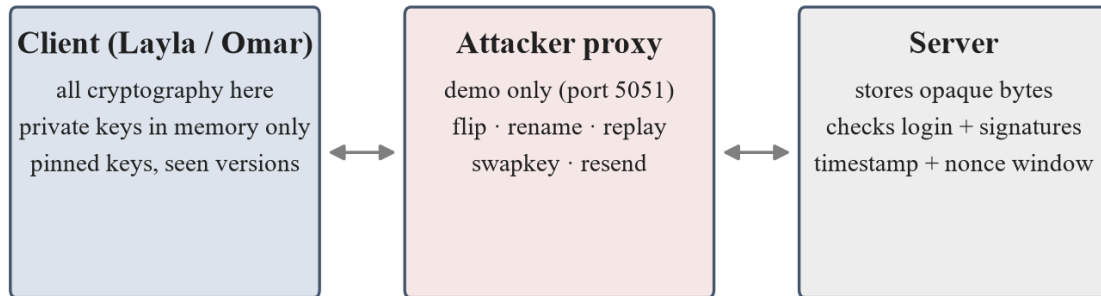
We defend against eavesdropping, modification of ciphertext, metadata, keys or signatures in transit, replay of requests and of documents, substitution of public keys (man/woman-in-the-middle), theft of the database followed by offline password guessing, and registered users who try to upload or share in someone else's name. Malware on a user's machine, coercion, traffic analysis of sizes and timing, denial of service and client-side side channels are out of scope.

System Design

- **Architecture**

SecureVault is a client and a server that talk over TCP. All cryptography happens in the client; the server stores opaque bytes, relays them, and checks login challenges and request

signatures. For the demonstration, a separate attacker proxy sits between them and can change any answer.



TCP sockets · length-prefixed binary frames · every request signed with Ed25519

Figure 1: SecureVault architecture with the attacker proxy used in the demonstration

• Key Hierarchy

Each layer locks only the one below it. Argon2id turns the password into a master secret, and HKDF splits it into two unrelated keys: `auth_key`, which the server holds for login, and `enc_key`, which never leaves the client and encrypts the user's X25519 and Ed25519 private keys. A random 256-bit data-encryption key (DEK) protects each document version and is wrapped separately for every recipient.

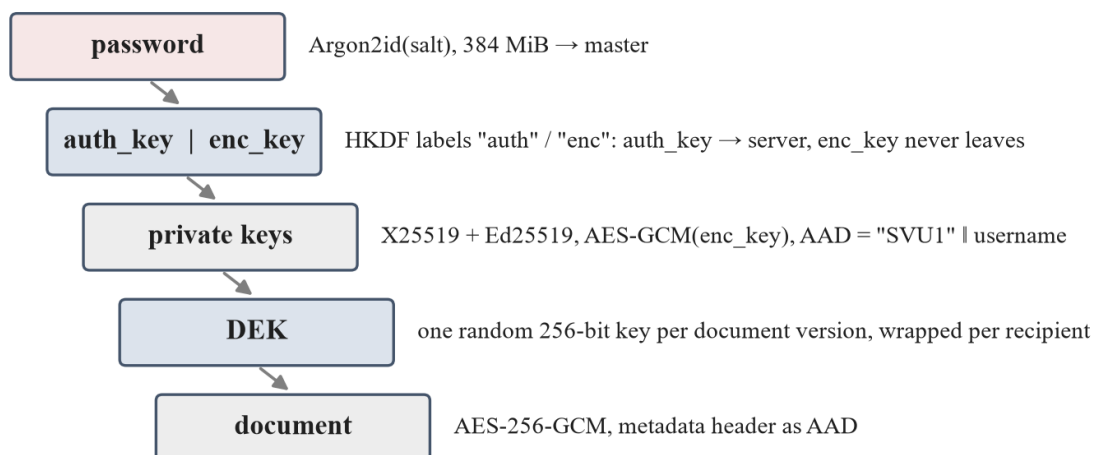


Figure 2: Key hierarchy from password to document

Because the password only locks the top layer, changing it re-encrypts one 92-byte key blob and nothing else; documents, grants and safety numbers stay valid. Sessions need no tokens:

every request after login is signed with Ed25519 and carries a timestamp and a nonce, and logging out simply wipes the unlocked keys from memory.

• Byte-Level Formats

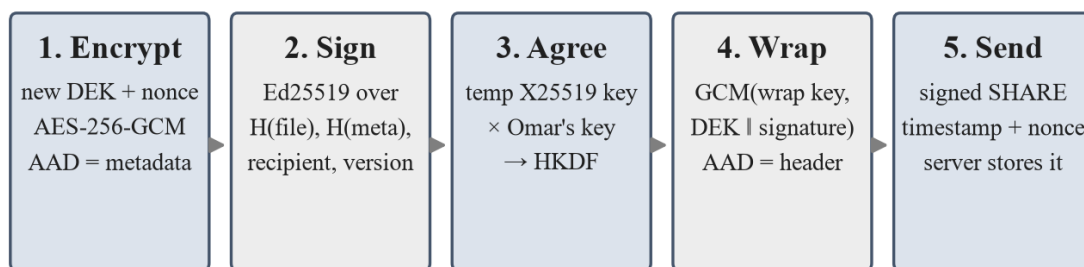
All integers are big-endian; LV denotes a 2-byte length followed by the value, so field boundaries can never be confused. Decoders reject truncated data, extra bytes and wrong type tags.

Table 2: Byte-level formats of stored objects and protocol messages

Object	Layout (bytes)	Authenticated as
Document	SVD1 doc_id 16 owner LV filename LV mime LV size 8 timestamp 8 version 4 nonce 12 length 4 + ciphertext tag 16	AAD = every field before the nonce
Key grant	SVK1 doc_id 16 version 4 sender LV recipient LV temp X25519 key 32 nonce 12 enc(DEK signature) 96 tag 16	AAD = every field before the nonce
Signed statement	SVS1 doc_id version sender LV recipient LV SHA-256(metadata) SHA-256(plaintext)	Ed25519 signature, never stored in the clear
User record	username LV salt 16 auth_key 32 X25519 pk 32 Ed25519 pk 32 key blob 92	key blob AAD = svu1 LV(username)
Request	length 4 type 1 username LV timestamp 8 nonce 16 body signature 64	Ed25519 over type ... body
Reply	length 4 status 1 payload	—

• Life of a Document

To upload, the owner generates a DEK and nonce, encrypts the file with AES-256-GCM using the metadata header as additional authenticated data, and sends the document with a grant addressed to herself in one signed request. The server checks the signature, the freshness window, ownership, and that the version is exactly one higher than before. Sharing never re-encrypts the file; it only produces a new grant, as shown in Figure 3.



The server only ever sees ciphertext, a temporary public key and a sealed 96-byte block.

Figure 3: Producing a key grant when Layla shares a document with Omar

On download, the recipient's client runs five checks in a fixed order and stops at the first failure. Nothing is displayed or written to disk until all five pass.

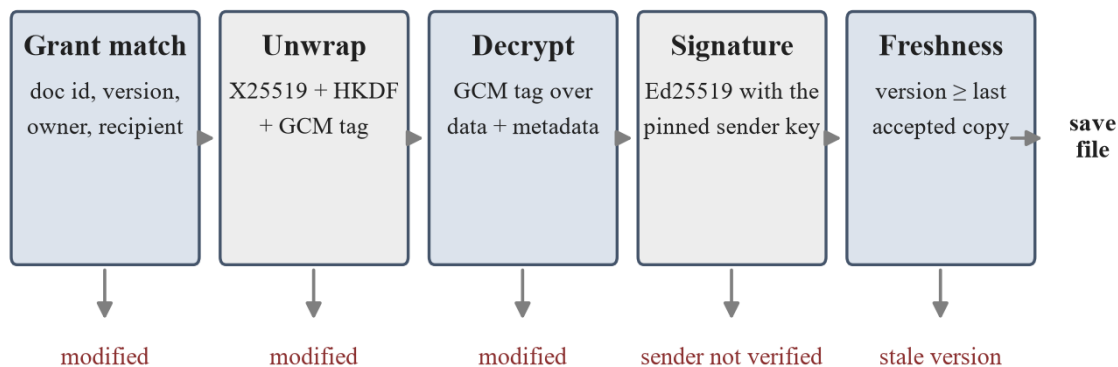


Figure 4: Verified retrieval: five ordered checks before a file is saved

Design Decisions and Justification

• Protecting Passwords

We use **Argon2id** (RFC 9106) with a 16-byte random salt per user and parameters $m = 384$ MiB, $t = 3$, $p = 4$. The salt is public because its job is uniqueness: equal passwords produce different records and each account must be attacked separately. The work factor was chosen by measurement (`tools/bench_password_hash.py`, median of five runs): 384 MiB is the size closest to our 0.5 s target, and we added memory rather than passes because memory bandwidth is what limits a GPU attacker.

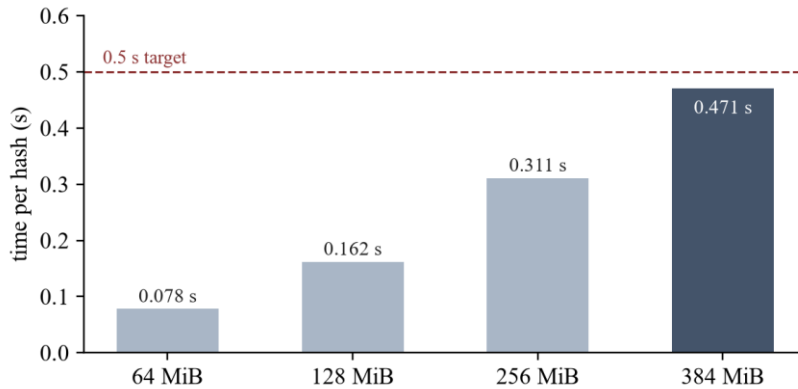


Figure 5: Measured Argon2id time per hash ($t = 3$, $p = 4$)

We rejected PBKDF2 (not memory-hard; SHA-256 ASICs are cheap), bcrypt (4 KiB of state, 72-byte password limit, not a KDF) and scrypt (memory and time are coupled). Argon2id comes from `argon2-cffi` and is our declared library component: our own pure-Python attempt needed 3.6 s for only 8 KiB, so a from-scratch version would have forced a memory cost too small to matter.

- **Protecting Document Contents**

Documents are encrypted with our own **AES-256-GCM**. A nonce can never repeat under one key because every DEK encrypts exactly one message, even across crashes and restarts. The metadata is the AAD, so the server can read it but any change breaks the tag, and the tag is verified before any plaintext is released. AES-256 sits above our 128-bit target by design: it costs four extra rounds and adds margin when many keys are in use. We rejected composing CBC with HMAC ourselves (more keys, and the order must be exactly right), ChaCha20-Poly1305 (sound, but outside the course material) and CCM (two passes, no benefit).

- **Public-Key Cryptography**

Users share no secret beforehand and the server must not know one, so public-key cryptography does two jobs: **X25519** (RFC 7748) delivers each DEK, and **Ed25519** (RFC 8032) signs documents. Curve25519 was chosen for resistance to implementation mistakes: every 32-byte string is a valid X25519 key, so there is no invalid-curve check to forget, and Ed25519 derives its per-signature nonce deterministically, which rules out the ECDSA nonce-reuse failure. Both offer about 128-bit security, equal to RSA-3072. Figure 6 quantifies the advantage we claim.

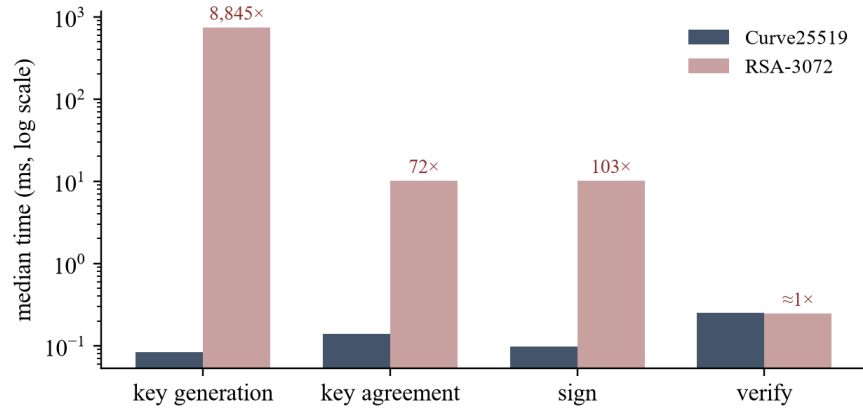


Figure 6: Curve25519 versus RSA-3072 at 128-bit security (OpenSSL, median of 20 runs)

Keys shrink from 384 to 32 bytes and signatures from 384 to 64 bytes. RSA verification is as fast as ours because of its small public exponent; this does not change the choice, since every sign-up and every share needs key generation, key agreement and signing.

- **Trusting a Public Key**

A dishonest server could hand Layla its own key instead of Omar's. We combine trust-on-first-use with a safety number: the first key fetched is pinned, both clients display the same 20-digit number computed with SHA-256 from both names and both key pairs, and the users compare it by phone or in person. **Sharing is refused until the recipient is verified**, which closes the first-contact gap of plain TOFU, and a pinned key that changes is blocked without an override. Twenty digits (about 66 bits) rather than twelve prevent an attacker from grinding keys until the numbers collide. A certificate authority run by the server was rejected because the server is the untrusted party.

- **Signatures Versus MACs**

GCM tags on documents and grants provide integrity; an Ed25519 signature over the statement in Table 2 provides origin and non-repudiation. A MAC cannot give non-repudiation however long its tag, because the recipient holds the same key and could have produced it. The recipient's name is inside the statement so Omar cannot forward the document as if it had been sent to someone else, and the signature travels encrypted inside the grant so the server cannot test guesses about a file's contents against it.

- **Freshness**

A replayed object is genuine, so signatures alone cannot stop it. Requests carry a signed timestamp (accepted within five minutes) and a 16-byte nonce that the server remembers for that window; documents carry a signed version number, and each client stores the highest version and the exact fingerprint it accepted, reporting anything older as stale; logins answer a fresh single-use 32-byte challenge with `HMAC(auth_key, challenge)`.

- **Architecture, Formats and Errors**

A socket architecture was chosen over a shared folder because it lets a real proxy control the network, as the threat model states. When verification fails, the client shows one of three short messages (document was modified, sender could not be verified, stale version), never reveals which byte or check failed, and never reports the failure to the server. A failed login always returns the same text; for unknown users the server returns a stable fake salt, `HMAC(server secret, username)`, and performs the same work so that timing does not reveal whether an account exists.

Security Argument

Table 3: Security properties and the mechanisms that provide them

Property	Mechanism
Confidentiality	AES-256-GCM with a per-document DEK, wrapped with X25519 + HKDF; the server never holds <code>enc_key</code> or a private key
Integrity	GCM tag over ciphertext and metadata, checked before decryption
Data-origin authentication	Ed25519 signature checked with the sender's pinned key
Non-repudiation	The same signature, verifiable by anyone with the file, metadata and signature (<code>prove</code> and <code>tools/verify_proof.py</code>)
Credential protection	Argon2id at 384 MiB with a per-user salt; only <code>auth_key</code> is stored
Freshness	Signed timestamp and nonce, single-use login challenge, signed version and fingerprint
Metadata binding	Metadata is the AAD and is hashed into the signature; the grant header is the grant's AAD

- **How the Mechanisms Fit Together**

The properties hold only because of the order in which they are applied. Authentication precedes decryption and every check precedes any output. The plaintext is signed first and the

signature is then encrypted, which keeps third-party proofs possible while hiding the signature from the server. Every context is bound: the recipient in the signature, both parties in the grant AAD and the username in the key-blob AAD, so a valid piece moved into another context fails. Keys are separated by purpose, and freshness is decided last so that an attacker cannot make a forged object look merely old.

- **Attacks Tried Against Our Own System**

Table 4: Attacks run against SecureVault over real sockets

Attack (via the proxy)	Result
Flip one ciphertext byte	document was modified
Rename the file in the metadata	document was modified
Replay an old version after a newer one	stale version
Swap a public key on first contact	safety numbers differ
Swap a public key that is already pinned	Blocked: key changed
Resend a captured request to the server	Request rejected

Offline guessing is quantified in Table 5 (`tools/crack_estimate.py`). Each Argon2id guess moves about 3.4 GiB of memory, so a high-end GPU with about 1 TB/s of bandwidth is limited to roughly 276 guesses per second, with only 64 fitting in its memory at once. The GPU figures are assumptions; times are averages (half the space searched) for a single account.

Table 5: Estimated offline attack time per stolen account

Password space	Argon2id (ours)	Bare SHA-256
8 characters, a–z and digits	≈ 160 years	≈ 1 minute
8 characters, letters and digits	$\approx 12,500$ years	≈ 1.4 hours
10 characters, letters and digits	≈ 48 million years	≈ 220 days
One of the 1,000,000 most common	≈ 30 minutes	instant

Implementation and Testing

SHA-256, SHA-512, HMAC, HKDF, AES-256, GCM, X25519, Ed25519, the safety number, the formats and the protocol were written by the team; only Argon2id comes from a library, as justified above. Operating-system randomness, sockets and storage are not cryptographic components. Correctness is established two ways: official vectors (FIPS 180-4, NIST GCM,

RFC 5869, 7748, 8032 and 9106) catch a wrong algorithm, and random cross-checks against the cryptography library catch edge cases the vectors miss, such as partial blocks, empty inputs and padding boundaries. The suite contains 235 tests, including system tests of the formats, key hierarchy, sharing, server, client and the attacker.

As bonus work, a deliberately weakened build shows that a reused nonce lets one known file reveal another and that skipping the tag check turns "Transfer 100" into "Transfer 900", while our real build rejects both. A graphical client presents the safety number in large groups and requires the user to type the last group they heard, keeps failure dialogs open for three seconds with no "open anyway" option, and lists only verified contacts when sharing.

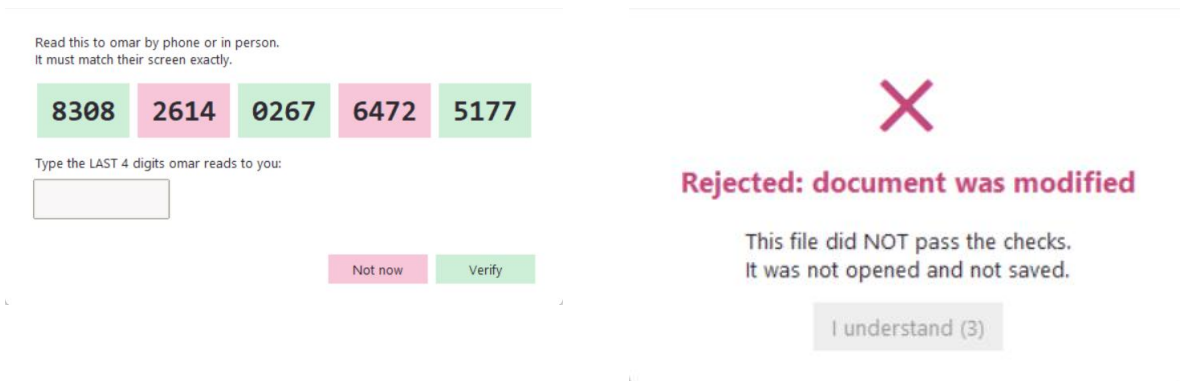


Figure 7: Graphical client: safety-number check and a rejected download

Limitations

- A password from a common word list still falls in about 30 minutes; a strength check at sign-up would help.
- The safety number protects only users who actually compare it.
- A stolen database yields `auth_key`, which allows logging in to the server but not reading files or signing; signature-based login would remove this.
- A forgotten password loses the private keys and every document; a printed recovery key would address this.
- A password change does not rotate keys, so an old database copy plus a cracked old password still opens them.
- The server sees file names, sizes, times and who shares with whom, and account names are not secret outside failed logins.

- The pure-Python code is not constant-time and GCM runs at about 105 KiB/s; neither is acceptable in production.
- Access to a shared version cannot be revoked.

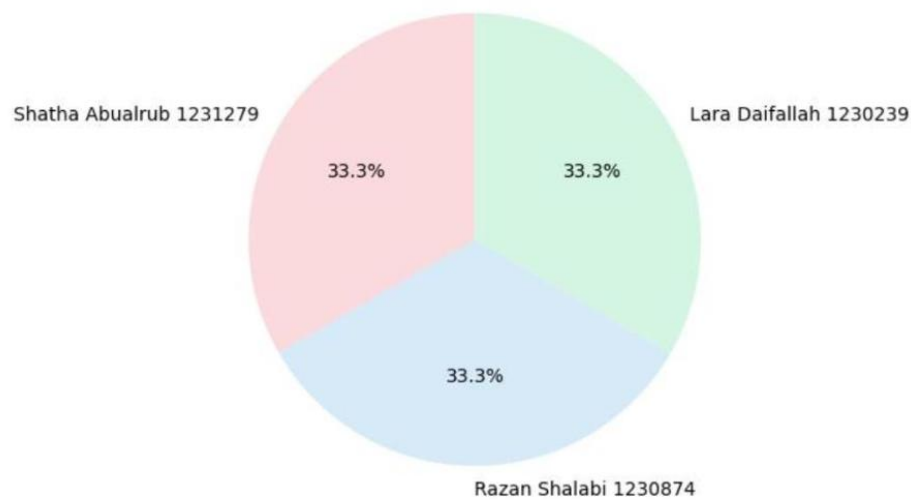
AI Usage Statement

We used an AI assistant as a support tool during the practical part of this project. We made the security design and decisions ourselves based on understanding and after deep searching of options and their pros and cons: which primitives to use, the key hierarchy, the trust model and the threat analysis. We relied on the AI mainly for parts that are hard to handle alone within the project timeline:

1. **Low-level primitive details:** working through the large-number arithmetic in X25519 and Ed25519 (field operations modulo $2^{255} - 19$, point encoding) and the bit-level steps of SHA-256/512 and AES.
2. **Finding and reading references:** pointing us to the relevant standards (RFC 7748, RFC 8032, RFC 5869, NIST FIPS 180-4, FIPS 197, SP 800-38D) and their official test vectors, so we could check our implementations.
3. **Translating our design into Python:** helping us turn concepts we already understood into working code, such as byte encoding, socket framing and the structure of the client and server.
4. **Building the attack demonstrations:** setting up the attacker proxy and running the tamper, rename, replay and key-substitution scenarios in VS Code.
5. **Debugging and testing:** explaining error messages and helping us write unit tests against the standard test vectors.
6. **Key derivation and separation:** helping us implement HKDF-based key separation, password-derived key handling and per-document keys without accidentally reusing the same key for authentication, encryption and key wrapping.
7. **Replay and freshness protection:** assisting us with the interaction between timestamps, request nonces, login challenges and document version numbers, and with testing that an old but correctly signed message is still rejected.
8. **Cryptographic integration:** helping us connect Argon2id, AES-256-GCM, X25519 and Ed25519 correctly across registration, login, upload, sharing and download, especially the order of signing, encryption, authentication and verification.

Team Contributions

This project was completed through effective teamwork and close collaboration among all group members. The workload was carefully divided to ensure that each member contributed meaningfully to the overall design and implementation. Responsibilities were distributed across different aspects of the project.



Conclusion

SecureVault meets the scenario it was built for: the server stores every byte without being able to read any of it, recipients learn and can prove who sent each document, and every modification, replay and key substitution we attempted is rejected. The decisions behind it were made against named alternatives, supported by measurements, and checked by tests against official vectors, which leaves the limitations above as the honest boundary of what the design protects.

References

- [1] NIST, FIPS 180-4: Secure Hash Standard (SHA-256, SHA-512), 2015.
- [2] NIST, FIPS 197: Advanced Encryption Standard, 2001.
- [3] NIST, SP 800-38D: Galois/Counter Mode (GCM) and GMAC, 2007.
- [4] NIST, SP 800-57 Part 1: Recommendation for Key Management (security-strength equivalences).
- [5] H. Krawczyk et al., RFC 2104: HMAC; RFC 5869: HKDF.
- [6] A. Langley et al., RFC 7748: Elliptic Curves for Security (X25519).
- [7] S. Josefsson, I. Liusvaara, RFC 8032: Edwards-Curve Digital Signature Algorithm (Ed25519).
- [8] A. Biryukov et al., RFC 9106: Argon2 Memory-Hard Function.
- [9] OWASP, Password Storage Cheat Sheet.
- [10] hashcat, public GPU benchmark results (SHA-256 speed used in Table 5).
- [11] ENCS4320 lecture notes 05–08: Hash Functions; Integrity and Authenticated Encryption; Public-Key Cryptography; Elliptic Curve Cryptography.